



LangChain - OutputParsers

By 

LangChain - OutputParsers

- **Structured Output** → when the model supports native function(tool) calling
- **Output Parsers** → when the model does NOT support native structured output or you want manual

with_structured_output() - LangChain converts your schema into JSON Schema and sends it inside the API request to model as tool call section.

```
{
  "model": "gemini-2.5-flash",
  "messages": [...],
  "tools": [
    {
      "type": "function",
      "function": {
        "name": "MySchema",
        "parameters": {
          "type": "object",
          "properties": {
            "movie": {"type": "string"},
            "sentiment": {
              "type": "string",
              "enum": ["pos", "neg"]
            }
          }
        },
        "required": ["movie", "sentiment"]
      }
    }
  ],
  "tool_choice": "required"
}
```

Model → tool_call JSON → Pydantic → Object

With output parser, The schema is just text inside the prompt to model rather as tool call section.

```
{
  "model": "gemini-2.5-flash",
  "messages": [
    {
      "role": "user",
      "content": "Return JSON like this: {...}"
    }
  ]
}
```

Model → Text → Extract JSON → json.loads() → Pydantic → Object

LangChain - OutputParsers



Parser	Purpose	Needs get_format_instructions?
StrOutputParser	Return raw text	× Never
JSONOutputParser	Return JSON	⚠ Optional/Recommended
StructuredOutputParser	Return fixed-keys JSON	✓ Yes, Removed from latest versions
PydanticOutputParser	Strict Python schema	✓ Yes, only this to used for Strict schema

StrOutputParser

- StrOutputParser is the **simplest parser in LangChain**.
- It does **not enforce JSON**, **not enforce structure**, and **does not need format instructions**.
- It simply returns the model's output **as a string**.

Ex:

```
from langchain_core.output_parsers import StrOutputParser  
parser = StrOutputParser()
```

JSONOutputParser

Simplest parser — ensures the LLM returns **valid JSON**.

Use when:

- You only need a basic JSON object
- No strict schema
- Just want JSON → Python dict

Ex:

```
parser = JsonOutputParser()
```

```
template = PromptTemplate(
    template='Give me 5 facts about {topic} \n {format_instruction}',
    input_variables=['topic'],
    partial_variables = {'format_instruction': parser.get_format_instructions()}
)
```

```
chain = template | model | parser
```

Step1 : Template does below to prompt before sending it to model

Adds formatting instructions like: Below instructions are appended to the prompt.

“Return a valid JSON object. Do not include anything else.”

€ **Not mandatory** to use get_format_instructions() You can manually tell the model “Return JSON only”.

Step2 : Model gives JSON string not JSON object

Step3: JSONOutputParser “Internally parses the JSON string (similar to json.loads()) after extracting it from the model output.

get_format_instructions()

get_format_instructions() method injects **exact formatting rules** into the prompt.

- It forces the model to follow the schema, Without it, parsers will fail
 - ✓ StrOutputParser & JSONOutputParser's version is trivial
 - ✓ StructuredOutputParser & PydanticOutputParser meaningful schemas

Usage in PromptTemplate:

```
template = PromptTemplate(
    template="Give info about {topic} \n {format_instructions}",
    partial_variables={"format_instructions": parser.get_format_instructions()},
)
```

Why is it called partial?

- In LangChain's PromptTemplate, you have **two kinds of variables**:
 - ✓ **input_variables** (dynamic values)
 - ✓ **partial_variables** (static or pre-filled values)

Ex:

Give me 5 facts about {topic} \n {format_instruction}

After applying partial variables:

Give me 5 facts about {topic} <the JSON or schema instructions as per parser used>

- Still **one placeholder left** → {topic} → filled later at runtime of chain or invoke

StructuredOutputParser

- Used when you need a **strict JSON schema** and **is NOT available in LangChain 1.1.0 and repurposed through PydanticOutputParser**
 - **Use when:**
 - You want clear fields
 - Each field has descriptions
 - You want predictable structure
 - **How it works:**
 - You define **ResponseSchema** objects. This defines **individual key** of JSON schema with name and description for StructuredOutputParser.

Ex:

```
schemas = [
    ResponseSchema(name="summary", description="Short summary"),
    ResponseSchema(name="sentiment", description="pos or neg")
]
parser = StructuredOutputParser.from_response_schemas(schemas)
```

get_format_instructions() in the **PromptTemplate** generates a **detailed JSON schema**, e.g.:

The output should be a JSON object with:
summary: string – Short summary
sentiment: string – Sentiment (pos/neg)

PydanticOutputParser

- Used for **strongest, type-safe, class-based schemas**.
- **Use when:**
 - You want strict validation
 - You want Python dataclasses
 - You want typed fields (list[str], int, Literal, Optional, etc.)

```
class Review(BaseModel):
    summary: str
    sentiment: Literal["pos", "neg"]
    pros: Optional[list[str]] = None
```

```
pydantic_parser =
PydanticOutputParser(pydantic_object=Review)
```

```
chain = template | model | pydantic_parser
```

```
template = PromptTemplate(
    template='Generate the name, age and city of a fictional
    {place} person \n {format_instruction}',
    input_variables=['place'],
    partial_variables={'format_instruction':
pydantic_parser.get_format_instructions()
})
```

Step 1 – Prompt Preparation

Template adds:

- JSON schema instructions (auto-generated from Pydantic)

Step 2 – Model Returns JSON String

Model returns **text containing JSON**, not a Python object.

Step 3 – Parser Processing

Parser:

- 1.Extracts JSON from model text
- 2.Converts JSON string → Python dict (similar to json.loads())
- 3.Returns validated Pydantic object

get_format_instructions():

Generates **auto JSON schema** from Pydantic.

Benefits:

- Automatic schema creation
- Automatic type validation
- Automatic errors if invalid JSON



Thank you